

pdfexplore

pdfexplore is the read-only PDF companion to mdexplore, with the same UI shell and operational model where it is applicable to PDFs.

It is designed for people who review many PDFs and need stable annotation/session context without touching source documents. pdfexplore treats the PDF file as immutable input and stores user state in sidecars so workflows remain reproducible and reversible.

What This Tool Prioritizes

- Read-only handling of source PDFs.
- Fast directory navigation with interaction-first scheduling.
- Sidecar-based persistence for colors, view sessions, and text highlights.
- UI parity with mdexplore interaction patterns where they fit PDF workflows.

Experience Goals

- Predictable state continuity:
 - highlights, view tabs, and marker badges should persist through navigation, refreshes, and restarts.
- Interaction-first responsiveness:
 - scrolling, expanding folders, and selecting files should remain smooth even while background cache/scan work is active.
- Transparent metadata model:
 - all durable user state lives in sidecars that are easy to inspect and back up.

Features

- Left-pane tree rooted at the chosen directory, showing folders plus *.pdf files.
- Right-pane embedded PDF preview via bundled local pdf.js.
- Top actions:
 - ^ to move root up one directory.
 - Refresh to rescan the current tree root.
 - Add View to create another tabbed view of the same PDF at the current page/scroll location.
- F5 refresh shortcut (same behavior as Refresh).
- Search box and color-apply controls matching mdexplore search syntax:
 - supports tokenized/quoted boolean queries via shared parser helpers,
 - searches non-recursively within direct-child PDFs of the selected directory,
 - highlights in-tree match counts and in-view search hits for matched files.
- Right-rail search indicators in preview:
 - markers are generated progressively from top pages to bottom pages,
 - markers become clickable as soon as they appear,
 - dense nearby markers are clustered into larger pills for easier click targets,
 - clicking a marker temporarily prioritizes navigation/rendering over marker-build throughput,
 - marker generation resumes automatically after the interaction settles.
- Tree highlight colors and clear actions:
 - per-file color assignment via tree context menu,
 - Clear in Directory and Clear All with confirmation prompts,
 - persisted per-directory in sidecars.
- Copy controls aligned with mdexplore :
 - destination mode Clipboard or Directory ,
 - pin action copies current PDF,
 - color actions copy all files with a selected highlight color in active scope,
 - directory copy merges color/view/highlight metadata into destination sidecars.
- Preview context menu:
 - Copy Selected Text ,
 - Highlight / Highlight Important ,
 - Remove Highlight .
- Persistent in-view text highlights with per-file sidecar storage.
- View tabs aligned with mdexplore behavior:
 - per-document multi-view tabs (max 8),
 - tab position progress bar driven by page progress,
 - custom tab labels via tab context menu,
 - labeled-tab Return to beginning support,
 - home/reset icon actions from shared ViewTabBar ,
 - hide tab strip when only one unlabeled default view remains,
 - per-document tab sessions persisted/restored from .pdfexplore-views.json .

- PDF preview widget caching:
 - once a PDF preview widget is created, it remains cached in memory for the app run,
 - switching away and back avoids full viewer/widget re-instantiation.
- Idle-aware text prefetch + cache badges:
 - current open PDF is prioritized for text-cache warming,
 - visible-scope PDFs are prefetched in small batches,
 - prefetch backs off during active interaction (especially scrolling),
 - cached-text badge reflects both memory and disk cache hits.
- Tree marker badges:
 - view badge indicates multi-view/open-session state,
 - marker badge indicates persistent text highlights,
 - marker sync merges scan results with live/in-memory metadata to reduce badge flicker.
- If the currently previewed PDF changes on disk, preview auto-refreshes and reports that in the status bar.
- `Refresh` preserves expanded folders/selection when possible and clears the preview if the active PDF was deleted.
- Effective root persisted to `~/ .pdfexplore.cfg` on close (interactive terminal launch without path defaults to current working directory).

User Workflow (Narrative)

1. Choose a root directory (or launch on a single PDF).
2. Navigate tree folders and open PDFs in the preview panel.
3. Use search to identify matching PDFs in visible scope.
4. Apply file highlight colors or preview text highlights as needed.
5. Add extra views (tabs) for parallel reading positions in one document.
6. Copy files and merge metadata to another directory when curating output sets.
7. Close and reopen later with session and highlight context restored from sidecars.

Architecture Overview

- Main GUI orchestration: `pdfexplore/app.py` (`PdfExploreWindow`).
- Tree model/delegate: `pdfexplore/tree.py` + shared `mdexplore_app/file_tree.py` .
- Background jobs: `pdfexplore/workers.py` .
- Embedded viewer bridge: `pdfexplore/assets/viewer_bridge.js` + local `pdf.js` bundle.
- Text caching:
 - memory cache: bounded `OrderedDict` ,
 - disk cache: compressed entries under `${XDG_CACHE_HOME:-~/ .cache}/mdexplore/pdfexplore-text-cache` .

Data/Control Flow Summary

- Tree selection/open action:
 - GUI updates active file/tab state,
 - preview widget is reused or created,
 - viewer bridge restores view state and highlights.
- Search action:
 - visible candidate set is built,
 - workers evaluate query in chunks,
 - model receives hit counts and filename match styling.
- Search-indicator action (viewer bridge):
 - term signature is computed from normalized query + current document key,
 - per-page text extraction reuses document-scoped in-memory cache entries,
 - pages are scanned in bounded concurrent batches,
 - marker entries are published progressively after each batch,
 - periodic event-loop yields keep input/paint responsive,
 - click interactions can interrupt and resume builds to keep navigation immediate.
- Prefetch action:
 - idle timer evaluates interaction/search pressure,
 - current document may be prioritized,
 - cache-hit/miss results feed badge state.
- Marker update action:
 - sidecar scan worker emits marker sets,
 - app merges scan state with live state,
 - model receives final marker path keys for rendering.

Run

From repo root:

```
./pdfexplore.sh [PATH]
```

- `PATH` may be a directory or a single PDF file.
- If omitted:
 - interactive terminal launch uses current working directory,
 - otherwise falls back to `~/pdfexplore.cfg`, then `$HOME`.

Configuration

- Primary config file: `~/ .pdfexplore.cfg`.
- Supports modern JSON payload with default root/recent roots.
- Includes compatibility handling for legacy single-path config payloads.
- Recent roots are maintained using short lock-based writes for safer multi-instance behavior.

Sidecars

`pdfexplore` writes sidecars beside PDFs/directories:

- `.pdfexplore-colors.json`
- `.pdfexplore-views.json`
- `.pdfexplore-highlighting.json`

`views` sidecars store multi-view sessions (tabs, active tab, per-tab state, custom labels when present). As with `mdexplore`, default untouched single-view sessions are not kept as persistent entries.

Sidecar Compatibility and Safety

- Sidecar parsing is defensive: malformed entries are ignored where possible.
- Legacy and partial shapes are tolerated in key paths (especially views).
- Sidecars are the only durable writes; source PDFs are never edited.

Sidecar Quick Reference

- `.pdfexplore-colors.json`
 - file highlight colors for tree rows.
- `.pdfexplore-views.json`
 - multi-view tab sessions and active view placement.
- `.pdfexplore-highlighting.json`
 - persistent in-document text highlight ranges (normal/important).

Performance Notes

- Search is worker-based and cancellation-aware.
- Prefetch is low-priority and throttled by interaction detection.
- Tree marker scans run in background workers and are merged with live state.
- Viewer search-marker generation uses document-scoped page-text cache plus bounded concurrency.
- Marker builds publish partial rails progressively so long documents become navigable early.
- Marker clicks are interaction-prioritized: active builds can pause briefly so page jumps feel immediate.
- Temporary reduced-paint mode may be used during tree mutation bursts, but marker sync forces full marker visibility when marker state is updated.

Performance Tradeoffs

- Throughput is intentionally sacrificed during active interaction periods to keep UI feel stable.
- Prefetch uses small batches to avoid large contiguous extraction spikes.
- Marker sync work is root-scoped to reduce unnecessary model churn after root changes.

What “Idle Prefetch” Means Here

- The app is considered idle enough when no recent interaction pressure or active search scan is blocking prefetch.
- Prefetch warms text cache for likely-next PDFs and updates cache badges progressively.

Troubleshooting

- If browsing feels stalled after a root change, verify that no long-running search query is active.
- If marker/cache badges look stale, trigger **Refresh** ; badge state is rebuilt from sidecars plus in-memory live state.
- If a sidecar gets malformed, browsing still works; affected metadata (colors/views/highlights) for that directory may be ignored until fixed.

Badge Troubleshooting Checklist

If highlight or cache badges seem wrong:

1. Confirm sidecar files exist beside the relevant PDFs/directories.
2. Trigger **Refresh** and re-open the affected file.
3. Verify active root/scope matches where metadata was created.
4. Re-run search and clear search to force model icon state refresh.

Crash/Freeze Triage Hints

- Repro patterns involving tree expansion and preview switching often indicate scheduling contention between marker sync and prefetch/search workers.
- Repro patterns involving white preview flashes often indicate viewer-bridge or preview-widget lifecycle timing.

Notes

- PDF content remains read-only; sidecars are the only persisted metadata outputs.
- The embedded viewer bundle is local under `pdfexplore/vendor/pdfjs/` (no remote dependency for core PDF viewing).

Contributor Notes

- Keep behavioral parity with `mdexplore` where users expect consistency.
- Prefer shared helper reuse from `mdexplore_app` for generic tree/model concerns.
- Update `pdfexplore/AGENTS.md` when architecture contracts or major workflow guarantees change.